

Week 4: An Introduction to Flutter

Dr M AOUE

Lecture notes

Objectives. By the end of this lecture you will know what a widget is and the difference between stateless and stateful widgets; you will be able to create a Flutter project and run it; and you will build a first Flutter app step by step with `MaterialApp`, `Scaffold`, `AppBar`, `Text`, a custom widget, `Column` and `Row`.

Every step of the lecture is a complete Flutter app. The files are in the `lib` folder of the project `flutter_examples`, inside `code/week04`. The listings below are taken directly from those files, and automated tests check that each app shows what is described here. See the guide *Running the programs in VS Code* to run them. The second part of this week, *Flutter Architecture and Widget Types*, is a separate document.

1 What is Flutter?

- Flutter is a collection of **widgets**, and **everything in Flutter is a widget**.
- A widget is a Dart class that represents a component of the user interface, such as an icon or a button.
- Every widget extends one of two classes: `StatelessWidget` or `StatefulWidget`.

Stateless widgets

A stateless widget extends `StatelessWidget`. Its state is fixed: it cannot change at run time. To show something different, a parent widget must build a new widget. Examples: `Text`, `Icon` and the buttons.

Stateful widgets

A stateful widget extends `StatefulWidget`. It can change its state (its contents) dynamically while the app runs, and redraws itself when `setState` is called. Examples: `Checkbox`, `Radio` and `TextField`.

The example below shows both kinds: `Greeting` is stateless (it only shows its name), `AcceptTerms` is stateful (it remembers whether the checkbox is ticked).

```

1 import 'package:flutter/material.dart';
2
3 // ----- Stateless: the content never changes by itself
4 -----
5 class Greeting extends StatelessWidget {
6   final String name; // fixed input: set once, never modified
7   const Greeting({super.key, required this.name});
8
9   @override
10  Widget build(BuildContext context) {
11    return Text('Hello, $name');
12  }

```

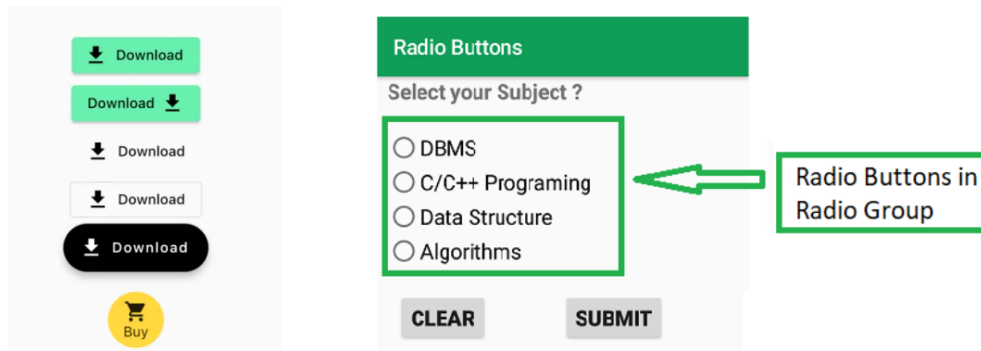


Figure 1: Stateless widgets: text, icons and buttons (left). Stateful widgets: radio buttons, which remember which one is selected (right).

```

12 }
13
14 // ----- Stateful: the content changes at run time -----
15 // The widget class is immutable; the changing data lives in its State object.
16 class AcceptTerms extends StatefulWidget {
17   const AcceptTerms({super.key});
18
19   @override
20   State<AcceptTerms> createState() => _AcceptTermsState();
21 }
22
23 class _AcceptTermsState extends State<AcceptTerms> {
24   bool _accepted = false; // the data that changes
25
26   @override
27   Widget build(BuildContext context) {
28     return Row(
29       mainAxisAlignment: MainAxisAlignment.min,
30       children: [
31         Checkbox(
32           value: _accepted,
33           // setState changes the data AND makes Flutter call build() again.
34           onChanged: (value) => setState(() => _accepted = value ?? false),
35         ),
36         Text(_accepted ? 'Accepted' : 'Not accepted'),
37       ],
38     );
39   }
40 }
41
42 // ----- Demo app (only used when this file is run) -----
43 void main() => runApp(const StatelessStatefulApp());
44
45 class StatelessStatefulApp extends StatelessWidget {
46   const StatelessStatefulApp({super.key});
47
48   @override
49   Widget build(BuildContext context) {
50     return const MaterialApp(
51       home: Scaffold(
52         body: Center(
53           child: Column(
54             mainAxisAlignment: MainAxisAlignment.min,
55             children: [Greeting(name: 'Rami'), AcceptTerms()],
56           ),
57         ),
58       ),
59     );

```

```

58     },
59     );
60 }
61 }

```

2 Creating a Flutter project

We create a simple “hello world” application.

With Android Studio

Start Android Studio and choose **File** → **New** → **New Flutter Project**. Check that the Flutter SDK path is correct and click **Next**. Type **week4** as the project name, choose the folder where the project is created, and click **Finish**.

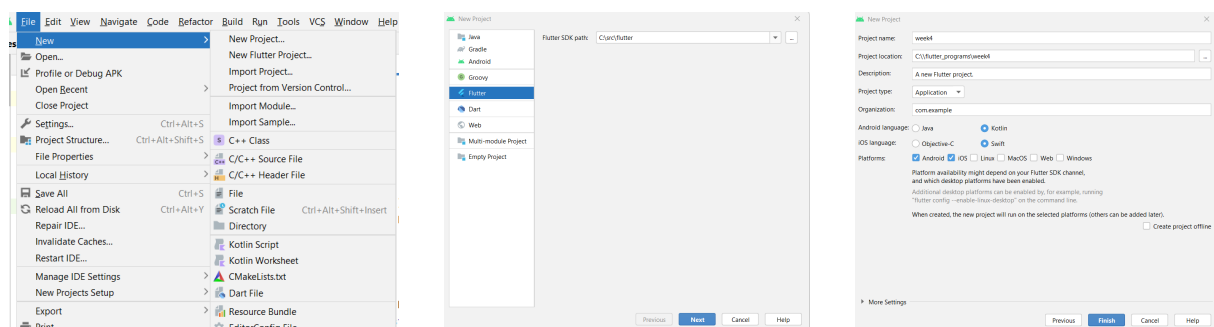


Figure 2: New Flutter Project in Android Studio: the menu, the SDK path, and the project name.

With VS Code

Open the Command Palette (**Ctrl+Shift+P**), run **Flutter: New Project**, choose **Application**, then choose a folder and a name. Or type in a terminal:

```
flutter create week4
```

In both cases Flutter creates a project that contains a sample application (a counter). The code of your app is in the file `lib/main.dart`.

3 Creating an emulator and running the app

To test the app on an Android phone you need a device: an emulator or a real phone.

- **Android Studio:** open the **Device Manager** and click **Create device**. When the device is created, click its **Launch** icon.
- **VS Code:** you can create the emulator in Android Studio’s Device Manager, or run the app on **Chrome** or on **Windows (desktop)**, which need no emulator. Choose the device at the bottom right of the VS Code window.

Then click the **Run** icon (in VS Code: **Run** above `main`, or **F5**). The sample application appears in the emulator: tap the **+** button to increase the counter.

4 The Hello World application

Now we create our own application. We replace all the code of `main.dart` with the code below, and then improve it step by step. Each step is one file of the project.

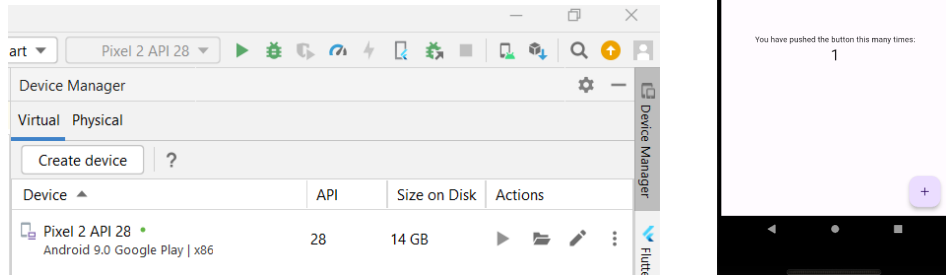


Figure 3: The Device Manager with an emulator (left), and the sample application running in it (right).

Step 1: the smallest app

```

1 import 'package:flutter/material.dart';
2
3 // Start the app: MyApp is the root widget. The lambda form is the same as
4 // void main() { runApp(const MyApp()); }
5 void main() => runApp(const MyApp());
6
7 class MyApp extends StatelessWidget {
8   // Older tutorials write: const MyApp({Key? key}) : super(key: key);
9   // "super.key" is the shorter form that current Flutter versions use.
10  const MyApp({super.key});
11
12  // build() describes what this widget looks like.
13  @override
14  Widget build(BuildContext context) {
15    // A widget that fills the screen with a color.
16    return Container(color: Colors.white);
17  }
18 }

```

- `import 'package:flutter/material.dart';` gives access to the widgets we need.
- Since this is Dart code, we define a `main` function. We use the lambda notation (week 2) to call `runApp`.
- `runApp` takes a widget and attaches it to the screen. By default the widget fills the whole screen. We pass it our `MyApp` widget.
- The constructor is called with `const`: `MyApp` is a stateless widget, so its state does not change at run time, and `const` makes the app run faster.
- `MyApp` is a stateless widget: the contents of the screen will not change. In Android Studio, type `st` and choose `stless` to create this class.
- Its `build` method returns the widget to display. An empty `Container()` shows a black screen, so we give it a color, for example white.

Step 2: displaying text

To display text we use a `Text` widget. We must wrap it in a `MaterialApp`, and we wrap the text in a `Center` to put it in the middle of the screen.

```

1 class MyApp extends StatelessWidget {
2   const MyApp({super.key});
3
4   @override
5   Widget build(BuildContext context) {
6     return MaterialApp(
7       // "home" is the widget shown on the screen.
8       home: Center(
9         // "child" is the widget that Center places in the middle.
10        child: Text('Hello World'),
11      ),
12    );
13  }
14 }

```

- **MaterialApp** gives the app the Material Design look. It has optional fields; **home** is the widget shown on the screen.
- By default the text would be displayed in the top left corner: **Center** places its **child** in the middle.
- Careful: **MaterialApp** alone gives the text an ugly style: red letters with a yellow underline. It is a warning style that tells the developer the app has not defined a default text style.

Step 3: adding a Scaffold

The warning style disappears when the text is inside a **Scaffold**, which gives the page a better background and text color. We use the **body** field of the **Scaffold** for the main content.

```

1 class MyApp extends StatelessWidget {
2   const MyApp({super.key});
3
4   @override
5   Widget build(BuildContext context) {
6     return MaterialApp(
7       home: Scaffold(
8         // "body" is the main content of the Scaffold.
9         body: Center(child: Text('Hello World')),
10      ),
11    );
12  }
13 }

```

Step 4: adding an AppBar

The **appBar** field of the **Scaffold** takes an **AppBar**. Setting **centerTitle** to **true** centers its title.

```

1 class MyApp extends StatelessWidget {
2   const MyApp({super.key});
3
4   @override
5   Widget build(BuildContext context) {
6     return MaterialApp(
7       home: Scaffold(
8         appBar: AppBar(
9           title: Text('Home Page'),
10          centerTitle: true, // put the title in the middle of the bar
11        ),
12        body: Center(child: Text('Hello World')),
13      ),
14    );
15  }
16 }

```

```

14     );
15   }
16 }

```

Step 5: changing the text style

The style field of Text takes a TextStyle: here a font size of 24 and the color deep purple.

```

1 class MyApp extends StatelessWidget {
2   const MyApp({super.key});
3
4   @override
5   Widget build(BuildContext context) {
6     return MaterialApp(
7       home: Scaffold(
8         appBar: AppBar(title: Text('Home Page'), centerTitle: true),
9         body: Center(
10          child: Text(
11            'Hello World',
12            style: TextStyle(fontSize: 24, color: Colors.deepPurple),
13          ),
14        ),
15      ),
16    );
17  }
18 }

```

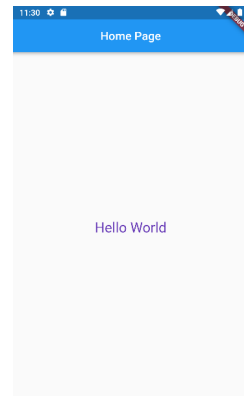
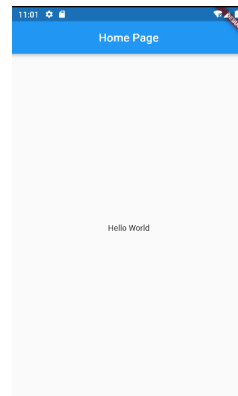
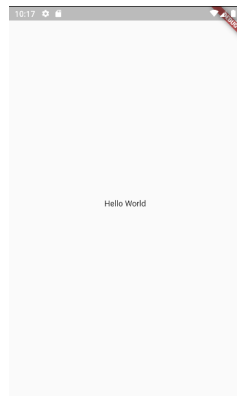


Figure 4: The result of steps 2 to 5, from left to right: MaterialApp alone, with a Scaffold, with an AppBar, and with a TextStyle.

Step 6: a custom widget

We can turn the Text into our own widget class, and reuse it whenever we need a similar text. In Android Studio, right-click the Center widget, choose **Refactor** and then **Extract Flutter Widget**; name it MyTextWidget and click **Refactor**. In VS Code, put the cursor on the widget, press Ctrl+. and choose **Extract Widget**.

Flutter replaces the body with MyTextWidget(). We then modify the widget to accept the text as a parameter: we add a field text and make it a required parameter of the constructor. Now the text is chosen when the widget is created.

```

1 // The custom widget: it shows any text in the purple style.
2 class MyTextWidget extends StatelessWidget {
3   final String text; // we add a field called text
4   // ...and make it a required parameter of the constructor

```

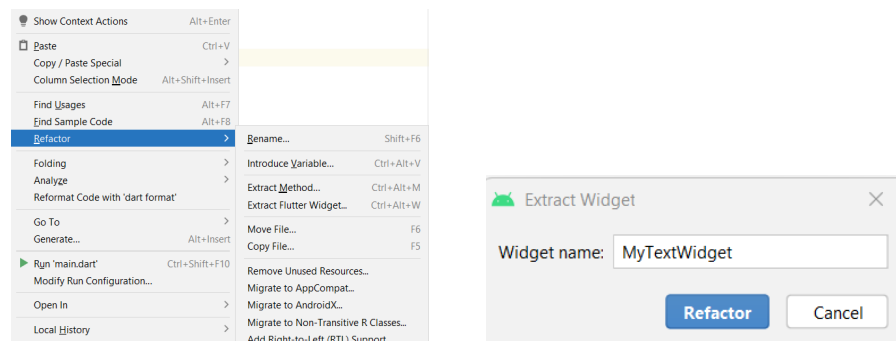


Figure 5: Extract Flutter Widget in Android Studio: the Refactor menu and the name dialog.

```

5  const MyTextWidget({required this.text, super.key});
6
7  @override
8  Widget build(BuildContext context) {
9    return Center(
10     child: Text(
11       text, // the field replaces the fixed 'Hello World'
12       style: TextStyle(fontSize: 24, color: Colors.deepPurple),
13     ),
14   );
15 }
16
17
18 class MyApp extends StatelessWidget {
19   const MyApp({super.key});
20
21   @override
22   Widget build(BuildContext context) {
23     return MaterialApp(
24       home: Scaffold(
25         appBar: AppBar(title: Text('Home Page'), centerTitle: true),
26         // now we can pass any text to the widget
27         body: MyTextWidget(text: 'Text 1'),
28       ),
29     );
30   }
31 }

```

Step 7: a Column

Column places widgets vertically. The widgets are given as a list in its children field. A SizedBox with a height of 16.0 adds empty vertical space between them.

```

1  class MyApp extends StatelessWidget {
2    const MyApp({super.key});
3
4    @override
5    Widget build(BuildContext context) {
6      return MaterialApp(
7        home: Scaffold(
8          appBar: AppBar(title: Text('Home Page'), centerTitle: true),
9          body: Column(
10             // The widgets of the column, from top to bottom.
11             children: <Widget>[
12               SizedBox(height: 16.0), // 16 logical pixels of empty space
13               MyTextWidget(text: 'Text 1'),
14               SizedBox(height: 16.0),

```

```

15         MyTextWidget(text: 'Text 2'),
16     ],
17 ),
18 ),
19 );
20 }
21 }

```

Step 8: a Row

Row aligns widgets horizontally. Here a Row with three texts is added to the Column, and `MainAxisAlignment.spaceEvenly` spreads them evenly.

```

1 class MyApp extends StatelessWidget {
2   const MyApp({super.key});
3
4   @override
5   Widget build(BuildContext context) {
6     return MaterialApp(
7       home: Scaffold(
8         appBar: AppBar(title: Text('Home Page'), centerTitle: true),
9         body: Column(
10          children: <Widget>[
11            SizedBox(height: 16.0),
12            MyTextWidget(text: 'Text 1'),
13            SizedBox(height: 16.0),
14            MyTextWidget(text: 'Text 2'),
15            SizedBox(height: 16.0),
16            // A Row nested inside the Column.
17            Row(
18              // Spread the three children evenly across the width.
19              mainAxisAlignment: MainAxisAlignment.spaceEvenly,
20              children: <Widget>[
21                MyTextWidget(text: 'Text 3'),
22                MyTextWidget(text: 'Text 4'),
23                MyTextWidget(text: 'Text 5'),
24              ],
25            ),
26          ],
27        ),
28      ),
29    );
30  }
31 }

```

5 Logical pixels

In the previous steps we used a `SizedBox` with a height of 16.0: it creates a vertical space of 16 **logical pixels**.

- A logical pixel is the scaling unit of Flutter.
- A logical pixel may be several physical pixels, depending on the screen of the device: it can be 10 physical pixels on one device and 20 on another. The ratio is the *device pixel ratio*.
- So a widget keeps the same size with respect to different screens.

The following app shows the size of your screen in both units, using `MediaQuery`:

```

1 import 'package:flutter/material.dart';
2
3 void main() => runApp(const LogicalPixelsApp());

```

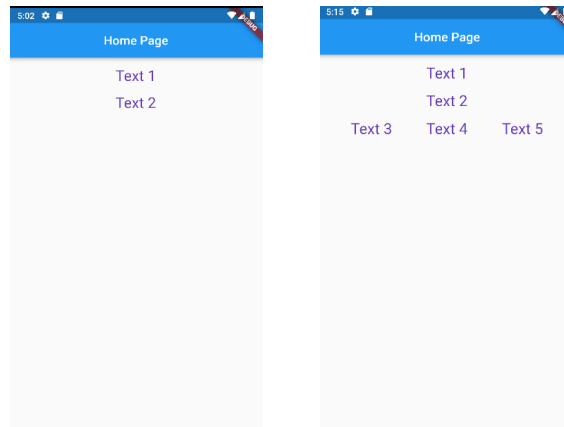


Figure 6: The result of step 7 (a Column, left) and step 8 (a Row in the Column, right).

```

4
5 class LogicalPixelsApp extends StatelessWidget {
6   const LogicalPixelsApp({super.key});
7
8   @override
9   Widget build(BuildContext context) {
10    return MaterialApp(
11      home: Scaffold(
12        appBar: AppBar(title: const Text('Logical pixels')),
13        body: const Center(child: ScreenFacts()),
14      ),
15    );
16  }
17 }
18
19 class ScreenFacts extends StatelessWidget {
20   const ScreenFacts({super.key});
21
22   @override
23   Widget build(BuildContext context) {
24     // Size of the screen (or window) in LOGICAL pixels.
25     final size = MediaQuery.sizeOf(context);
26     // How many physical pixels one logical pixel is.
27     final ratio = MediaQuery.devicePixelRatioOf(context);
28
29     return Column(
30       mainAxisAlignment: MainAxisAlignment.min,
31       children: [
32         Text('Logical size: ${size.width} x ${size.height}'),
33         Text('Device pixel ratio: $ratio'),
34         Text('Physical size: ${size.width * ratio} x ${size.height * ratio}'),
35         // The 16 of SizedBox(height: 16.0) is also logical pixels.
36         Text('16 logical pixels = ${16 * ratio} physical pixels'),
37       ],
38     );
39   }
40 }

```

6 Going further: three more building blocks

The steps below are not in the slides. They extend the app of the lecture with widgets and options that you will use in almost every app. Each one is again a complete app in the project (files 10, 11 and 12).

More AppBar options

The AppBar has more fields than `title` and `centerTitle`:

- `leading`: a widget at the left end of the bar, often a menu icon;
- `actions`: a list of widgets at the right end, usually icon buttons;
- `backgroundColor` and `foregroundColor`: the color of the bar, and the color of its title and icons;
- `elevation`: the size of the shadow under the bar (0 is flat).

An `IconButton` is a tappable icon: its `onPressed` field is a function that runs when the user taps it (see week 2). Here it shows a short message at the bottom of the screen with a `SnackBar`.

```

1 class AppBarOptionsPage extends StatelessWidget {
2   const AppBarOptionsPage({super.key});
3
4   // Shows a short message at the bottom of the screen.
5   void _say(BuildContext context, String message) {
6     ScaffoldMessenger.of(context)
7       .showSnackBar(SnackBar(content: Text(message)));
8   }
9
10  @override
11  Widget build(BuildContext context) {
12    return Scaffold(
13      appBar: AppBar(
14        title: const Text('AppBar options'),
15        backgroundColor: Colors.teal,
16        foregroundColor: Colors.white, // color of the title and the icons
17        elevation: 4, // a shadow under the bar (0 would be flat)
18        // leading: a widget at the left end of the bar
19        leading: IconButton(
20          icon: const Icon(Icons.menu),
21          onPressed: () => _say(context, 'Menu tapped'),
22        ),
23        // actions: widgets at the right end, in a list
24        actions: [
25          IconButton(
26            icon: const Icon(Icons.search),
27            onPressed: () => _say(context, 'Search tapped'),
28          ),
29          IconButton(
30            icon: const Icon(Icons.settings),
31            onPressed: () => _say(context, 'Settings tapped'),
32          ),
33        ],
34      ),
35      body: const Center(child: Text('Tap the icons in the app bar')),
36    );
37  }
38 }

```

More Scaffold features

A `Scaffold` has several slots, each filled with one widget: `appBar`, `body`, `bottomNavigationBar`, and a `backgroundColor` for the page. It expands to fill the whole screen, and it adjusts by itself when the on-screen keyboard appears. Use one `Scaffold` per page, as its top-level structure: do not put a `Scaffold` inside another one.

The example also uses a `Card` (a rectangle with rounded corners and a small shadow, used to group content) and `Padding`, which adds space around its child.

```

1 class MyApp extends StatelessWidget {
2   const MyApp({super.key});
3
4   @override
5   Widget build(BuildContext context) {
6     return MaterialApp(
7       home: Scaffold(
8         appBar: AppBar(title: const Text('Scaffold extras')),
9         // The color behind everything else on the page.
10        backgroundColor: Colors.blueGrey.shade50,
11        body: const Center(
12          child: Card(
13            child: Padding(
14              padding: EdgeInsets.all(24), // 24 logical pixels on every side
15              child: Text('A Card in the body'),
16            ),
17          ),
18        ),
19        // A bar at the bottom of the page.
20        bottomNavigationBar: const BottomAppBar(
21          child: Center(child: Text('Bottom bar')),
22        ),
23      ),
24    );
25  }
26 }

```

Sharing space with Expanded

A `SizedBox(height: 16.0)` has a fixed size, the same on every screen. Sometimes you want a widget to use the space that is *left*. Inside a `Column` (or a `Row`), an `Expanded` widget makes its child fill the space that the other children do not use.

The `flex` field says how the free space is shared: three `Expanded` widgets with `flex 1`, `2` and `1` receive one quarter, one half and one quarter of it. You never compute a size yourself; Flutter does it on any screen.

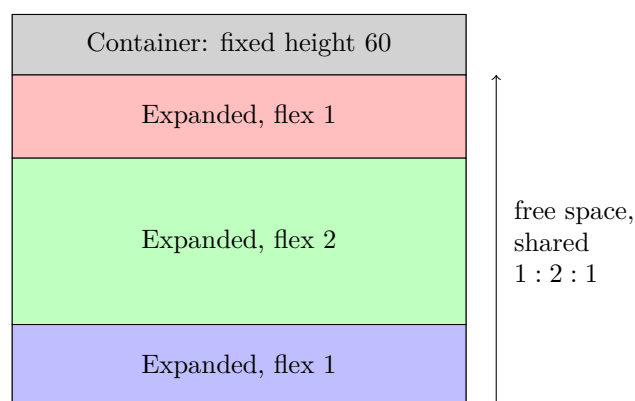


Figure 7: A `Column` on the screen: the fixed strip keeps its height, and the three `Expanded` widgets share the rest.

```

1 class ExpandedDemo extends StatelessWidget {
2   const ExpandedDemo({super.key});
3
4   @override
5   Widget build(BuildContext context) {
6     return Column(

```

```

7      children: [
8        // Fixed size: always 60 logical pixels tall.
9        Container(
10         key: const ValueKey('fixed'),
11         height: 60,
12         color: Colors.grey.shade400,
13         alignment: Alignment.center,
14         child: const Text('Fixed height: 60'),
15       ),
16       // The three Expanded widgets share the REST of the height: 1 : 2 : 1.
17       Expanded(
18         flex: 1,
19         child: Container(
20           key: const ValueKey('box-a'),
21           color: Colors.red.shade200,
22           alignment: Alignment.center,
23           child: const Text('Expanded, flex 1'),
24         ),
25       ),
26       Expanded(
27         flex: 2,
28         child: Container(
29           key: const ValueKey('box-b'),
30           color: Colors.green.shade200,
31           alignment: Alignment.center,
32           child: const Text('Expanded, flex 2'),
33         ),
34       ),
35       Expanded(
36         flex: 1,
37         child: Container(
38           key: const ValueKey('box-c'),
39           color: Colors.blue.shade200,
40           alignment: Alignment.center,
41           child: const Text('Expanded, flex 1'),
42         ),
43       ),
44     ],
45   );
46 }
47 }

```

A common mistake: a `ListView` (a scrolling list) placed directly inside a `Column` fails with the error “Vertical viewport was given unbounded height”. Wrap the `ListView` in an `Expanded`: it then takes the rest of the screen. You will use this when we study lists.

7 Widgets at a glance

Widget	Purpose
Container	a box that can be given a color and a size
MaterialApp	the root of an app that uses Material Design (title, theme, home page)
Scaffold	the basic structure of a page: app bar, body, and more
AppBar	the bar at the top of a page, with a title
Center	centers its child widget
Text	shows a line of text; style takes a TextStyle
Column	places its children vertically
Row	places its children horizontally
SizedBox	an empty box, used for a fixed space between widgets
IconButton	a tappable icon; onPressed runs when it is tapped
Card	a rectangle with rounded corners and a small shadow, to group content
Padding	adds space around its child
BottomAppBar	a bar at the bottom of a page (the bottomNavigationBar slot)
Expanded	inside a Column or Row : fills the free space; flex sets the share

This document is maintained by Dr. Mohamad AOUDÉ
Last updated: 2026-09-21